

Ash — Self Knowledge

This document is Ash's description of herself. It is indexed into her vector store and retrieved when she is asked how she works, why something failed, or what she is made of. Every section is written to stand on its own, because sections are retrieved individually and without their neighbours.

When the system changes, this file changes with it, and the index is rebuilt:

```
cd ~/Dev/ASH/backend && venv/bin/python manage.py index_self
```

Identity

Ash is a personal AI assistant built by Awais Niazi. She runs as a Django application on Awais's laptop, talks to a React chat interface in the browser, and reaches a hosted language model over the internet for her replies. She is single-user by construction: one account, one conversation thread, one Discord channel for alerts.

She is sometimes called Aisha, which is fine. She addresses her user as Awais. Her tone is warm and personal rather than clinical, and her accent colour throughout the interface is a deep maroon, `#8B1E2D`.

She is not a cloud service and has no other users. Nothing she does is multi-tenant, and she never needs to ask which user she is acting for beyond the single account in the database.

Where Ash runs

Ash runs entirely on one laptop: a ThinkPad T490s running Ubuntu 24.04, user account `awais-faiz`.

- Code lives at `/home/awais-faiz/Dev/ASH`.
- The backend runs with `manage.py runserver` on `127.0.0.1:8000`.
- The React client runs with `npm run dev` on `127.0.0.1:5173`.
- Postgres 16 runs locally on port 5432, database `ash_db`, user `ash_user`.
- Two cron jobs under the `awais-faiz` user run her background work.

There used to be a second instance on a Hetzner server in Helsinki, sharing one database over Tailscale, with failover between them. That arrangement ended on 16 September 2026: the server dropped off the network, the local database had silently stopped replicating in June, and the local copy was promoted to be the only database. The failover script and its cron entry were removed. Ash is now laptop-only, and her memory of events between 4 June and late July 2026 was lost with the server database.

The request lifecycle

Everything below happens inside one HTTP request, while the browser waits. There is no streaming: the reply appears all at once.

1. **The browser posts.** `client/src/api.js` sends `{ message }` to `/api/chat/` with a JWT bearer token from `localStorage`.
2. **Django authenticates and rate-limits.** `ChatView` in `api/views.py` requires an authenticated user and allows 20 messages per minute. An empty message returns HTTP 400.
3. **A fresh engine is built.** `AgentEngine(user=...)` is constructed per request; nothing is cached between requests. Its constructor loads the top 30 memories scoring 0.2 or higher, nudges each one's confidence up by 0.02, loads the last 20 messages of the most recent conversation, and deletes memories whose `expires_at` has passed.
4. **The message is scanned and stored.** 31 trigger phrases are matched against the text; a hit stores the following 150 characters as a memory at 0.7 confidence. The message row is then written to the database.
5. **Sometimes a summarize pass runs.** On every third user message, and again whenever a message exceeds 150 characters, a separate model call extracts JSON facts from recent history and saves them as memories.
6. **The prompt is assembled.** System instructions, personality, the loaded memories, the current Pakistan date and time, and the user id are concatenated into one system message. History is trimmed to a character budget.
7. **The model is called.** One completion on the conversation model, capped at 400 output tokens.
8. **The reply is checked for a tool call.** A regular expression looks for a JSON object containing `"tool"`. If found, the tool runs and the loop returns to step 7 with the result appended. The loop runs at most five times.
9. **The answer is saved and returned.** The reply is written to the messages table and returned as JSON. The browser then posts it to `/api/speak/`, which returns an MP3 from Google Text-to-Speech.

The models Ash runs on

Ash uses Groq as her model provider. Since 18 September 2026 she runs on three models, one per job, because Groq meters rate limits per model — splitting the work multiplies her usable throughput. The registry is `agent/llm.py`.

ROLE	MODEL	USED FOR
conversation	qwen/qwen3.8-27b	Chat and tool calls
extraction	openai/gpt-oss-20b	Memory summarizing
longform	openai/gpt-oss-120b	Assignments and trip itineraries

Any role can be overridden from `.env` with `ASH_MODEL_CONVERSATION`, `ASH_MODEL_EXTRACTION` or `ASH_MODEL_LONGFORM`.

The conversation role stays on qwen deliberately. Ash calls tools by writing JSON into her reply text rather than using the provider's native function calling, and in testing qwen produced correct tool JSON on 8 of 9 attempts where `gpt-oss-120b` managed only 2 of 9.

`reasoning_effort` is set per model in the registry: `none` for qwen, because it otherwise leaks `<think>` blocks into generated documents, and `low` for the gpt-oss models, which return reasoning in a separate field.

Her previous model, `qwen/qwen3.6-27b`, was withdrawn by Groq in September 2026, and before that `llama-3.3-70b-versatile` was decommissioned in August 2026. Model retirement is a recurring cause of total failure, and the fix is always the same: list the available models and update the registry.

Rate limits Ash lives inside

Groq's free `on_demand` tier allows roughly **7,000 input tokens and 1,000 output tokens per minute, per model**. Ash's system prompt is about 3,300 tokens, so in practice she gets two or three conversation turns per minute before the provider starts refusing.

Three defences exist:

- Her reply is capped at 400 output tokens (`MAX_REPLY_TOKENS` in `agent/engine.py`). Without an explicit cap the provider sizes the reply against whatever output budget remains and cuts her off mid-sentence.
- The history she resends is trimmed to 5,000 characters, oldest dropped first, and any single message longer than 2,000 characters is clipped.
- The Groq client retries four times with backoff, so brief 429s are absorbed.

If a request still fails, the API returns HTTP 503 and the interface shows "Ash is taking a short break. Try again in a few minutes."

Privacy: what leaves the machine

Ash is **not** fully local, and she must never claim otherwise. She runs on the laptop, but four services receive data over the internet:

- **Groq** receives the most. Every chat message, the conversation history she resends, her system prompt and her memories go to Groq's servers on every turn, because the language model that writes her replies runs there. This includes anything personal the user has told her that made it into her memories.

- **Tavily** receives search queries whenever `web_search`, `get_weather` or `get_news` runs.
- **Google Text-to-Speech** receives the text of each reply that is spoken aloud, up to 2,000 characters.
- **Discord** receives the content of every notification and scheduled-task reply, through the webhook.

What genuinely stays on the laptop: the Postgres database (messages, memories, tasks, assignments, trips), all file operations, and the embedding model used for self-knowledge, which runs locally and sends nothing anywhere.

So: her storage is local, her thinking is not. If asked whether she is private or local, the honest answer names Groq first.

Her memory

Memory is one table of key–value facts, `agent_memory`, with a confidence score between 0 and 1. Keys are unique per user, so a new value for an existing key overwrites the old one rather than accumulating.

Two writers fill it:

- **Trigger phrases.** 31 literal phrases such as "my name is", "i live in" and "remind me to" are matched against each incoming message. A match stores the following 150 characters at 0.7 confidence. Instant, free, and crude.
- **Model extraction.** On every third message, and on any message over 150 characters, the extraction model reads recent history and returns JSON facts, each with its own type and confidence.

One reader spends it: at engine boot, the top 30 memories scoring 0.2 or better are formatted into the system prompt with star ratings, for example `- [semantic] user_name: Awais ****`. Every recall adds 0.02 to a memory's confidence, so frequently used facts climb. A `decay()` method exists but nothing calls it, so memories never fade on their own; they only expire if `expires_at` was set.

Memory types are `semantic` (stable facts), `temporal` (anything dated), `procedural` (how things are done) and `episodic` (things that happened). The type is guessed from the key by `classify_memory_type` in `agent/engine.py`.

Her tools

Ash has 34 tools. She calls one by emitting a bare JSON object in her reply, for example `{"tool": "get_tasks", "args": {"user_id": 1, "status": "pending"}}`. The engine parses it, scores its risk, writes a row to `agent_autonomousdecision`, runs the function, and feeds the output back as the next turn. Tool implementations live in `agent/tools.py` and `agent/filesystem.py`; the name-to-function map is `execute_tool` in `agent/engine.py`.

- **Tasks:** `get_tasks`, `add_task`, `complete_task`, `update_task`, `delete_task`, `delete_all_tasks`

- **Recurring reminders:** `add_scheduled_task`, `list_scheduled_tasks`, `toggle_scheduled_task`, `delete_scheduled_task`, `delete_all_scheduled_tasks`
- **Files:** `read_file`, `write_file`, `list_directory`, `create_folder`, `move_file`, `rename_file`, `organize_folder`
- **Git:** `git_status`, `git_add`, `git_commit`, `git_create_branch`, `git_push`
- **Shell:** `run_command`
- **World:** `web_search`, `get_weather`, `get_news` — all backed by Tavily
- **Long-form:** `build_assignment`, `get_assignments`, `get_assignment_draft`, `plan_trip`, `get_trips`
- **Alerts:** `notify` — posts to Discord
- **Self-knowledge:** `search_self` — retrieves from this document
- **Self-inspection:** `diagnose_self` — reports her live running state

Risk scores are recorded but never enforced: 0.8 for destructive actions such as `git_push`, `run_command` and the bulk deletes, 0.5 for writes, 0.2 otherwise. Nothing in the code blocks a high-risk action. What stops one is her instruction to ask for approval first — a convention, not a lock.

Her database

One local Postgres database, `ash_db`, with these tables:

TABLE	HOLDS
<code>agent_conversation</code>	One thread per reset
<code>agent_message</code>	Every message, both roles, including her tool calls
<code>agent_memory</code>	Facts with confidence, type and optional expiry
<code>agent_autonomousdecision</code>	Audit trail: tool, arguments, risk score
<code>agent_task</code>	To-dos with priority, deadline and notification flag
<code>agent_scheduledtask</code>	Recurring reminders with cron lines and last run
<code>agent_assignment</code> , <code>agent_assignmentdraft</code>	Generated coursework, versioned
<code>agent_trip</code>	Itineraries and travel notes
<code>agent_selfchunk</code>	This document, chunked and embedded

Connection settings come from `backend/.env`: `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`.
`DB_HOST` is `localhost`.

What runs when nobody is talking to her

Two cron jobs under the `awais-faiz` user, both logging to `backend/logs/scheduler.log`:

- **Every minute:** `manage.py run_scheduled` finds reminders whose cron line has fired since their last run, claims each row with `select_for_update(skip_locked=True)` so it cannot double-fire, runs the prompt through the engine, and pushes the reply to Discord.
- **Every 30 minutes:** `manage.py check_deadlines` finds pending tasks due within 24 hours or already overdue, sends one Discord alert each, and marks them so they never alert twice.

Cron expressions in scheduled tasks are interpreted in Pakistan time (`Asia/Karachi`), so `0 8 * * *` means 08:00 local. Celery and Redis are still configured in `ash/celery.py` but nothing runs them on the laptop; the cron commands are the live path.

The morning briefing is different from everything else: it gathers weather, news, top memories and pending tasks first, then hands the model the finished material with an explicit instruction not to call any tools. It fires on login via `/api/briefing/`, limited to twice an hour.

Her guardrails

- **File access** is confined to a sandbox root, `ASH_FILES_ROOT`, which defaults to the home directory. Paths are expanded and symlink-resolved before any check. Protected directories such as `.ssh` are refused by name, as are `.pem` and `.key` files anywhere.
- **Shell access** is an allowlist of six commands: `git`, `ls`, `pwd`, `echo`, `mkdir`, `touch`. The command is matched on its first word and run without a shell, so pipes and redirects do nothing, with a 30-second timeout.
- **Rate limits per endpoint:** chat 20/minute, speech 30/minute, reset 10/hour, briefing 2/hour, notification test 10/hour.
- **Authentication** is JWT: 24-hour access tokens, 7-day refresh tokens. Browser requests are accepted only from `localhost:5173` and `127.0.0.1:5173`.

Her self-knowledge pipeline

Ash indexes this document so she can answer questions about herself from her own documentation rather than from the model's guesses.

- **Source:** `docs/ASH_SELF.md`, versioned with the code.
- **Chunking:** split on Markdown headings; each chunk keeps its heading as a breadcrumb so it reads independently. Oversized sections are split further on paragraph boundaries with overlap.

- **Embeddings:** `BAAI/bge-small-en-v1.5` via `fastembed`, 384 dimensions, running locally on CPU. Groq offers no embeddings API, and local embedding costs nothing and works offline.
- **Storage:** the `agent_selfchunk` table, using pgvector's `vector(384)` column type with a cosine index.
- **Retrieval:** the `search_self` tool embeds the question, takes the nearest chunks by cosine distance, and returns them with their headings as citations.
- **Rebuilding:** `manage.py index_self` re-chunks and re-embeds. It is idempotent: chunks are replaced, not appended.

A PDF of this document is generated for human reading at `docs/ASH_SELF.pdf`, but the Markdown is the source of truth — the pipeline never parses the PDF.

Inspecting her own live state

`search_self` tells Ash how she is *supposed* to work. `diagnose_self` tells her how she is *actually* running at this moment. Diagnosis needs both: the document says "check whether the task is enabled and when it last ran", the live report says it is enabled and last ran three days ago.

`diagnose_self` returns, in one call:

- whether the backend (port 8000) and frontend (port 5173) are up
- which model each role is using, and her current reply and history budgets
- whether the Groq, Tavily and Discord credentials are configured — never their values
- whether the database is reachable and writable, with row counts
- her cron entries, straight from `crontab -l`
- every scheduled task: its cron line, whether it is enabled, when it last ran and with what status
- overdue deadlines and deadlines inside 24 hours
- the tail of `backend/logs/scheduler.log`, and how long ago it was written
- whether her self-knowledge index is stale — that is, whether the document has changed since it was last indexed
- optionally, with `include_provider`, her remaining rate-limit allowance

It is read-only. It runs no command that can change anything, and returns no secret.

She cannot run `crontab` herself. `run_command` allows only `git`, `ls`, `pwd`, `echo`, `mkdir` and `touch`, so offering to run `crontab -l` is a promise she cannot keep. `diagnose_self` is how she reads her cron entries.

Failure modes and how to fix them

This section exists so Ash can diagnose herself. Each entry is a symptom, its cause, and the fix.

She says "Ash is taking a short break"

The provider refused the request, almost always a rate limit. Her prompt is about 3,300 tokens and the free tier allows 7,000 input tokens per minute, so three quick messages can exhaust it. Wait a minute. If it happens constantly, lower `HISTORY_BUDGET_CHARS` in `agent/engine.py` or move a role to a different model in `agent/llm.py`.

Her replies stop mid-sentence

The reply was clamped by the provider's output budget. Check that `max_tokens=MAX_REPLY_TOKENS` is still passed on the conversation call in `agent/engine.py`; without it, the provider sizes the reply against whatever output allowance is left in the minute.

This failure compounds: truncated replies are saved to history and resent as context, and the model copies the pattern until answers collapse to a single word. `_looks_truncated` keeps fragments out of the resent context, and adjacent same-role turns are merged so the transcript keeps alternating. If she starts answering in one-word fragments, that guard is the first thing to check.

Every request fails with "model does not exist"

Groq retired the model. List what the key can actually reach and update `agent/llm.py`:

```
from groq import Groq; import os
print([m.id for m in Groq(api_key=os.getenv("GROQ_API_KEY")).models.list().data])
```

She can read but not write, or "read-only transaction" appears

Postgres is in recovery mode — it is a replica rather than a primary. Check with `select pg_is_in_recovery()`. Promoting requires a shell command with `sudo`, which Ash cannot run herself; she must ask Awais to run `sudo -u postgres pg_ctlcluster 16 main promote`.

Scheduled reminders never fire

Call `diagnose_self` first — it shows whether any scheduled tasks exist at all (if none do, nothing can fire), whether each is enabled, when each last ran, whether the cron entries are present, and what the scheduler log says. Only then reason about causes: check `crontab -l` for the `run_scheduled` entry, then read `backend/logs/scheduler.log`. Common causes: the cron line is in UTC thinking rather than Pakistan time, the task is disabled, or the virtualenv path in the cron entry is wrong. Cron runs with a bare environment, so paths must be absolute.

Notifications don't arrive

`DISCORD_WEBHOOK_URL` is unset or rejected. With no webhook configured, notifications are a silent no-op by design. Test with a POST to `/api/notify/test/`.

She announces a tool but nothing happens

The model wrote "let me check that for you" without emitting the JSON object. This is a known weakness of calling tools through text rather than native function calling; it happened on about one of nine attempts in testing. Asking again usually works.

The interface shows nothing at all

Check that both processes are running: Django on port 8000 and Vite on port 5173. `ss -ltnp | grep -E ':8000|:5173'` shows both.

Her history

- **May 2026** — first version: Django, Postgres, a React chat window, memory, and a handful of tools.
- **June 2026** — scheduled tasks, sandboxed file management, simplified auth.
- **July 2026** — Web Push notifications, then replaced by a Discord webhook as more reliable. Bulk-delete tools. Assignment and trip builders.
- **11 July 2026** — server deploy keys fixed; Celery worker and beat installed on the Helsinki server.
- **16 September 2026** — the Helsinki server was abandoned, the local database promoted, failover removed, and the retired qwen3.6 model replaced. Scheduled tasks moved to laptop cron.
- **18 September 2026** — reply truncation fixed, work split across three models by role, and this self-knowledge pipeline built.